

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3
Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size
 F Filter Size
 P Padding
 S Stride

2.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

3. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

4. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

Total trainable parameters = 448.

6. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

7. Task 1: Dataset Loading Output

Training Images : (50000, 32, 32, 3)

Training Labels : (50000, 1)

Testing Images : (10000, 32, 32, 3)

Testing Labels : (10000, 1)

7.1 Sample Images



Figure 1: Sample images from the CIFAR-10 dataset.

7.2 Class Distribution

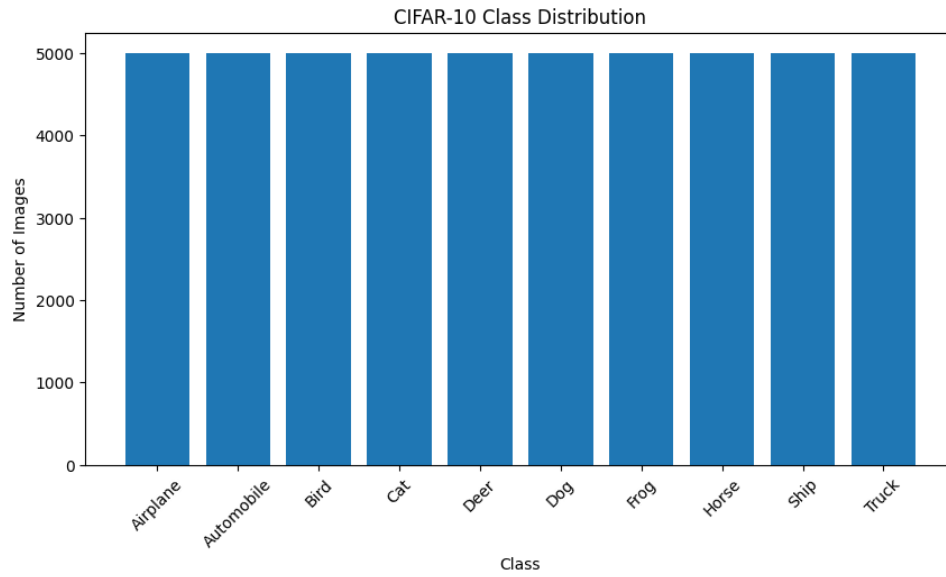


Figure 2: Distribution of classes in the training set.

8. Task 2: Convolution with Different Kernel Sizes

Original Image Shape : (1, 32, 32, 3)

3x3 Feature Map Shape : (1, 30, 30, 1)

5x5 Feature Map Shape : (1, 28, 28, 1)

7x7 Feature Map Shape : (1, 26, 26, 1)



Figure 3: Original sample image used for kernel-size comparison.

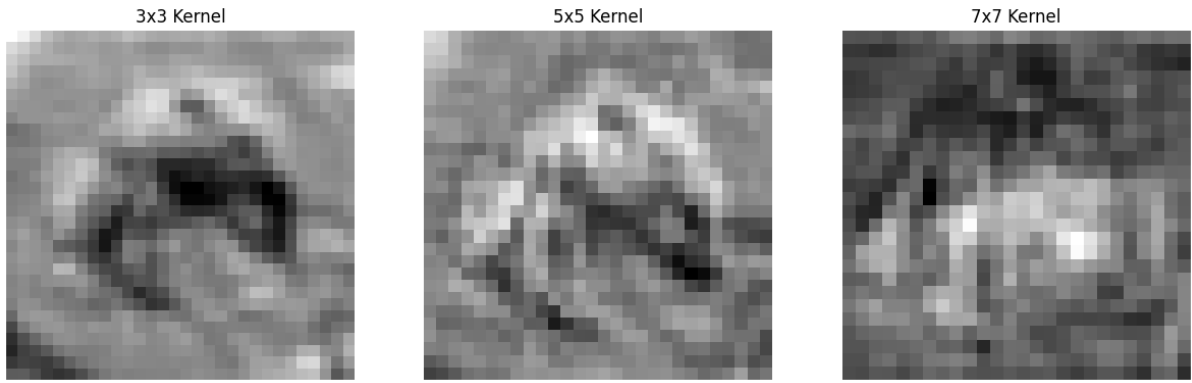


Figure 4: Feature maps produced using 3×3 , 5×5 , and 7×7 kernels.

9. Task 3: Stride and Padding Comparison

Input Shape : (1, 32, 32, 3)

Stride = 1, Padding = Valid : (1, 30, 30, 1)

Stride = 2, Padding = Valid : (1, 15, 15, 1)

Stride = 1, Padding = Same : (1, 32, 32, 1)

Stride = 2, Padding = Same : (1, 16, 16, 1)

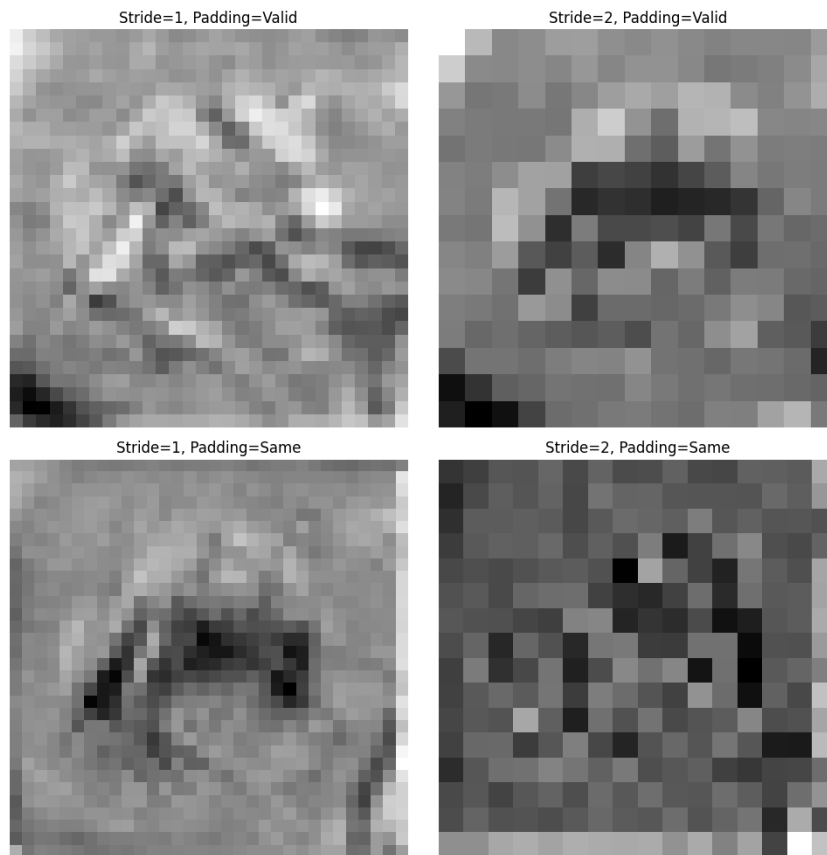


Figure 5: Feature maps for different stride and padding combinations.

10. Task 4: Feature Map Visualization

Feature Map Shape : (1, 32, 32, 8)



Figure 6: Original image used for feature map visualization.

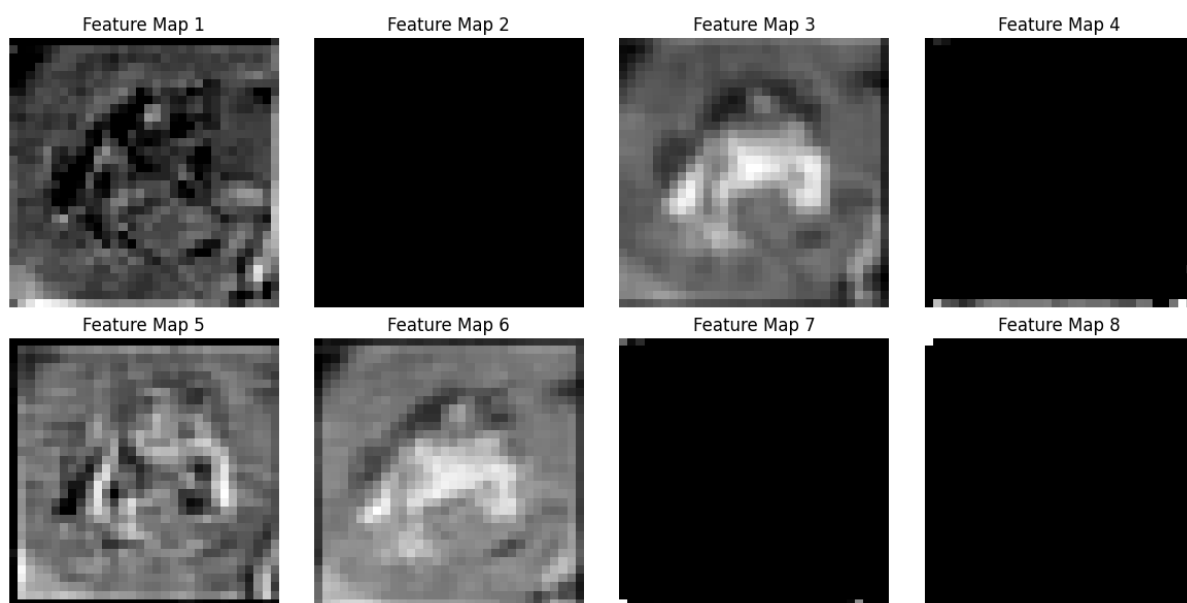


Figure 7: Eight feature maps generated by the first convolution layer.

Model summary (single Conv2D + MaxPooling2D, 8 filters):

Layer (type)	Output Shape	Param #
Conv2D	(None, 32, 32, 8)	224
MaxPooling2D	(None, 16, 16, 8)	0

Total params: 224 Trainable params: 224 Non-trainable params: 0

11. Task 6: CNN Architecture and Training

Model summary (full CNN):

Layer (type)	Output Shape	Param #
Conv2D	(None, 32, 32, 32)	896
MaxPooling2D	(None, 16, 16, 32)	0
Conv2D	(None, 16, 16, 64)	18,496
MaxPooling2D	(None, 8, 8, 64)	0
Flatten	(None, 4096)	0
Dense	(None, 128)	524,416
Dense	(None, 10)	1,290

Total params: 545,098 Trainable params: 545,098 Non-trainable params: 0

Training log (Optimizer: Adam, Epochs: 20, Batch Size: 32):

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.2063	2.1497	0.2874	2.0130
2	0.3037	1.9547	0.3218	1.9047
3	0.3447	1.8253	0.3476	1.8119
4	0.3748	1.7300	0.3826	1.7086
5	0.4003	1.6693	0.4022	1.6615
6	0.4130	1.6294	0.4108	1.6279
7	0.4255	1.5997	0.4237	1.6066
8	0.4354	1.5729	0.4363	1.5770
9	0.4465	1.5465	0.4420	1.5564
10	0.4554	1.5238	0.4532	1.5319
11	0.4631	1.5021	0.4583	1.5154
12	0.4696	1.4851	0.4558	1.5204
13	0.4769	1.4692	0.4709	1.4872
14	0.4811	1.4523	0.4718	1.4776
15	0.4864	1.4398	0.4761	1.4707
16	0.4907	1.4266	0.4626	1.5244
17	0.4958	1.4126	0.4923	1.4362
18	0.5009	1.4025	0.4963	1.4237
19	0.5051	1.3922	0.4993	1.4129
20	0.5104	1.3797	0.4974	1.4131

Table 1: Training log for the CNN in Task 6.

12. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Output Size:

The output size is calculated using the convolution formula and is 32×32 . This reduces the spatial dimensions while preserving important features.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**

The convolution layer has **1,792 trainable parameters**, including the weights and biases for all 64 filters.

3. **Compare ReLU and Sigmoid activation functions.**

ReLU is faster and commonly used in CNN hidden layers, while Sigmoid is mainly used for binary classification. ReLU also helps reduce the vanishing gradient problem.

4. **Replace Max Pooling with Average Pooling and compare the results.**

Average Pooling gives slightly better accuracy than Max Pooling in this experiment. Both methods provide similar performance overall

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

Increasing filters from 16 to 64 helps the model learn more complex features and can improve accuracy. However, it also increases computation time and memory usage.

13. Results

The results below are taken directly from the final (20th) training epoch of the CNN built in Task 6 (Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax):

- **Training Accuracy:** 51.04%
- **Validation Accuracy:** 49.74%
- **Training Loss:** 1.3797
- **Validation Loss:** 1.4131
- **Number of Parameters:** 545,098

Note: The uploaded notebook does not include a separate test-set evaluation (accuracy/precision/recall/F1/confusion matrix) or an Average-Pooling comparison run – Task 7 and Task 5 were not executed in the notebook, so those figures are not reported here.

14. Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?**

Convolution preserves the spatial structure of images and detects local features like edges and textures. It also uses fewer parameters than fully connected layers.

2. **How does stride affect the feature map size?**

A larger stride moves the filter by more pixels, resulting in a smaller feature map. A smaller stride produces a larger feature map

3. **What is the role of padding?**

Padding adds extra pixels around the image to control the output size. It also helps preserve information from the edges of the image.

4. **Why is pooling used?**

Pooling reduces the spatial dimensions of feature maps and decreases computation. It also helps the model focus on the most important features

5. **How do feature maps represent image characteristics?**

Feature maps show the features detected by convolution filters, such as edges, shapes, textures, and patterns. Different filters detect different characteristics of the image.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs use shared weights and local connections, so the same filter is applied across the image. This greatly reduces the number of trainable parameters compared to MLPs.

15. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.